

TP Séance 8

Contexte

À la fin de ce TP, vous aurez :

- **Séparé la logique métier** du DAG Airflow (S6) dans un module Python testable indépendamment, sans avoir besoin d'installer Airflow en CI.
- Écrit des **tests unitaires** avec `pytest` pour cette logique.
- Écrit un **workflow GitHub Actions** qui exécute automatiquement lint + tests à chaque `push`.
- Observé le pipeline **bloquer un déploiement** quand un test échoue.
- Observé le pipeline **déployer** (simulation) quand tout est vert.

Partie 0 : Récupérer les fichiers de référence et préparer la branche

Les fichiers de cette séance (`days/` , `tests/` , `requirements.txt` , et le workflow `.github/workflows/ci-anfa-seance08.yml`) sont **fournis prêts à l'emploi** sur le `main` du dépôt du cours. Synchronisez d'abord le `main` de votre fork avec celui du cours (`upstream`), puis créez votre branche de travail.

```
# 1. Mettre à jour le main de votre fork depuis le dépôt du cours
git checkout main
git fetch upstream
git merge upstream/main      # récupère seance-08/ + .github/workflows/
git push origin main         # pousse le main à jour sur votre fork

# 2. Créer votre branche de travail à partir de ce main
git checkout -b seance-08
cd seance-08
```

Note. `upstream` = le dépôt du cours, `origin` = votre fork. Si `upstream` n'est pas encore déclaré, faites-le une fois :
`git remote add upstream <URL-du-dépôt-du-cours>` (voir le README). Vous **ne poussez jamais** sur `upstream`.

À propos des snippets. Les fichiers étant fournis, ce TP **n'affiche que les passages clés** du code. Ouvrez les fichiers de `seance-08/` et `.github/workflows/` dans votre éditeur pour lire le code complet.

Important le workflow est à la racine. Le fichier `.github/workflows/ci-anfa-seance08.yml` vit à la racine du dépôt, pas dans `seance-08/` (obligatoire : GitHub Actions ne lit que ce chemin précis). Le `git merge upstream/main` l'a déjà déposé au bon endroit.

Point de vérification 0. Vous êtes sur `seance-08`, avec `seance-08/dags/`, `seance-08/tests/` et `.github/workflows/ci-anfa-seance08.yml` présents.

Partie 1 Séparer la logique métier du DAG

Problème. Le DAG `dag_anfa_quotidien.py` de la séance 6 mélange logique métier (vérifier des fichiers, construire un message) et objets Airflow (`PythonOperator` , `DAG`). Pour tester la logique métier en CI, on ne veut **pas** installer Airflow (lourd, lent). On extrait donc cette logique dans un module Python indépendant.

1.1 Le module de logique métier (fourni)

Le fichier `seance-08/dags/anfa_logic.py` est fourni. Ouvrez-le. Ses fonctions sont **pures** (pas de réseau, pas d'objet Airflow) testables sans rien installer d'autre que Python. Le cœur :

```
def verifier_liste_fichiers(objets: list) -> dict:
    if not objets:
        raise ValueError("Aucun fichier de résultat trouvé dans MinIO.")

    nb_fichiers = len(objets)
    taille_totale_octets = sum(o["Size"] for o in objets)

    return {
        "nb_fichiers": nb_fichiers,
        "taille_totale_ko": round(taille_totale_octets / 1024, 1),
    }
```

1.2 Le DAG qui utilise ce module (fourni)

Le fichier `seance-08/dags/dag_anfa_quotidien.py` est fourni : une version simplifiée de S6 qui **importe** la logique testée.

Partie 2 Les tests unitaires

2.1 Le fichier de tests (fourni)

Le fichier `seance-08/tests/test_anfa_logic.py` est fourni. Ouvrez-le : 5 tests, aucune dépendance à Airflow/boto3/MinIO.
Points clés :

```
import sys
import os

# Permet d'importer anfa_logic.py situé dans ../dags/
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "dags"))

import pytest # noqa: E402
from anfa_logic import ( # noqa: E402
    construire_cle_trajets,
    verifier_liste_fichiers,
    construire_message_notification,
)
```

À comprendre le `# noqa: E402`. On doit modifier `sys.path` avant d'importer `anfa_logic`, ce qui place ces imports après du code. `flake8` signalerait alors `E402: module level import not at top of file`. Le commentaire `# noqa: E402` dit à `flake8` d'ignorer cette règle sur ces lignes précises c'est volontaire et légitime ici. Sans lui, le lint de la CI échouerait avant même les tests.

Partie 3 Le workflow GitHub Actions (fourni)

Le fichier `.github/workflows/ci-anfa-seance08.yml` est fourni (à la racine du dépôt). Ouvrez-le. Il définit 2 jobs : `valider-dag` (lint + tests) puis `deployer` (simulation), ce dernier ne tournant que si le premier réussit.

```
on:
  push:
    branches: ["seance-08"]
    paths:
      - "seance-08/**"
  pull_request:
    branches: ["seance-08"]
    paths:
      - "seance-08/**"

jobs:
  valider-dag:
    name: Lint et tests unitaires
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: seance-08
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: flake8 dags/ tests/ --max-line-length=100
      - run: pytest tests/ -v

  deployer:
    name: Deployer le DAG valide (simulation)
    runs-on: ubuntu-latest
    needs: valider-dag
    if: github.event_name == 'push'
    defaults:
      run:
        working-directory: seance-08
    steps:
      - uses: actions/checkout@v6
      - run: |
          # deployment simple/dags
```

Partie 4 Déclencher le pipeline et observer

4.1 Commiter et pousser

```
cd ..  
git add seance-08/  
git commit -m "Séance 8 : pipeline CI/CD pour le DAG Anfa"  
git push -u origin seance-08
```

Le workflow étant déjà sur votre `main` (via le `merge upstream/main` de la Partie 0), un simple `push` sur `seance-08` suffit à le déclencher.

4.2 Observer dans l'onglet Actions de GitHub

Allez sur votre dépôt GitHub → onglet **Actions**.

Vous voyez le workflow « **CI Pipeline Anfa - Séance 8** » en cours (icône jaune), puis en succès (✅ vert) après 1 à 2 minutes. Cliquez dessus pour voir les deux jobs `valider-dag` et `deployer`.

Faites une capture d'écran du workflow réussi, avec les 2 jobs visibles elle ira dans votre RENDU.

Point de vérification 4. Le workflow s'est exécuté automatiquement après votre `push`, sans action manuelle sur GitHub.

Partie 5 Casser un test volontairement

Objectif : voir concrètement le pipeline bloquer un déploiement, comme il aurait dû bloquer le bug de Mawuli dans la situation-problème du CM.

5.1 Introduire un bug

Modifiez `seance-08/dags/anfa_logic.py`, fonction `verifier_liste_fichiers` :

```
return {  
    "nb_fichiers": nb_fichiers,  
    "taille_totale_ko": round(taille_totale_octets / 1000, 1), # ← BUG : 1000 au lieu de 1024  
}
```

5.2 Pousser le bug

```
git add seance-08/dags/anfa_logic.py  
git commit -m "Bug volontaire : mauvaise conversion Ko"  
git push
```

5.3 Observer l'échec

Retournez sur l'onglet **Actions**. Le job `valider-dag` échoue (✗) : le test

`test_verifier_liste_fichiers_calcule_correctement` échoue, car `3.0` (attendu) ≠ `3.1` (obtenu avec `/1000`, arrondi à 1 décimale).

Partie 6 Rédiger le RENDU et soumettre

Un modèle `seance-08/RENDU.md` est fourni. Ouvrez-le et complétez-le : nom, identifiant GitHub, résumé, réflexion personnelle, et insérez vos 2 captures.

```
mkdir -p seance-08/captures
# déposer les 2 captures (ci-succes.png, ci-echec.png)
cd ..
git add seance-08/
git commit -m "Séance 8 : RENDU et captures"
git push
```

Soumettez sur Google Classroom : `https://github.com/<votre-username>/<nom-de-votre-fork>/tree/seance-08`

Point de vérification final. Votre branche `seance-08` contient les dags, les tests, le RENDU et les captures ; le workflow tourne dans l'onglet Actions.

Contenu attendu de votre rendu

```
<nom-de-votre-fork>/    (branche seance-08)
├── .github/
│   └── workflows/
│       └── ci-anfa-seance08.yml    (fourni, à la racine)
├── seance-08/                    (fourni + complété par vous)
│   ├── RENDU.md
│   ├── requirements.txt
│   ├── dags/
│   │   ├── anfa_logic.py
│   │   └── dag_anfa_quotidien.py
│   ├── tests/
│   │   └── test_anfa_logic.py
│   └── captures/
│       ├── ci-succes.png
│       └── ci-echec.png
```

Pièges fréquents et solutions

Symptôme	Cause probable	Solution
Le workflow ne se déclenche pas	Le fichier n'est pas à la racine, ou aucun fichier de <code>seance-08/</code> n'a changé	Vérifier <code>.github/workflows/ci-anfa-seance08.yml</code> à la racine ; le filtre <code>paths</code> exige un changement dans <code>seance-08/</code>
<code>flake8 : E402 module level import not at top of file</code>	Les <code># noqa: E402</code> ont été supprimés des imports du fichier de tests	Remettre <code># noqa: E402</code> sur les lignes <code>import pytest</code> et <code>from anfa_logic import</code>
La CI est verte mais <code>deployer</code> est "Skipped"	Vous avez ouvert une PR au lieu de pousser	<code>deployer</code> a <code>if: github.event_name == 'push'</code> : normal sur une PR
<code>pytest</code> échoue en local mais pas en CI (ou l'inverse)	Cache <code>__pycache__</code> obsolète après édition	<code>rm -rf **/__pycache__ .pytest_cache</code> puis relancer

Pour aller plus loin (optionnel, hors évaluation)

- Ajoutez un job `construire-image` qui construit une image Docker contenant le DAG (voir le slide CM correspondant).
- Remplacez le déclencheur par une vraie **Pull Request obligatoire** : configurez une règle de protection de branche qui empêche le merge tant que la CI n'est pas verte.
- Ajoutez un test qui vérifie que `dag_anfa_quotidien.py` s'importe sans erreur (nécessite d'installer `apache-airflow` en CI plus lent).

Fin du TP de la séance 8.

Votre pipeline Anfa est maintenant **livré avec discipline** : testé avant chaque déploiement, tracé, réversible. La séance 9 répond à la question qui suit naturellement : une fois en production, **comment savoir si tout va bien ?** C'est le rôle du monitoring avec Prometheus et Grafana.